Plan de pruebas de la aplicación Bicis
Historia de usuario: Listar las Estaciones de una Red de Bicicletas

Historia de usuario. Yo, como usuario, quiero ver la lista de estaciones de una red de bicicletas, con las bicis disponibles y totales de cada una y si tiene bicis eléctricas, ordenada de más a menos bicis disponibles, de manera que pueda saber rápidamente en qué estación es más fácil coger una bici.

Interfaz esperada: mock-up de la historia (Docs/Mockups/US532377-ListarEstaciones.pdf), que muestra la pantalla "Estaciones de la red" con una cabecera (nombre de la red y "ciudad, país · N estaciones"), la lista de estaciones, un guion para los datos no disponibles y los mensajes de red sin estaciones y de error de carga. Las pruebas de aceptación aprobadas por el PO prevalecen sobre el mock-up en tres puntos: las bicis se muestran en el formato [bicis_disponibles]/[bicis_totales] (por ejemplo, "5/12"), las bicis eléctricas se indican con un icono y la lista se ordena por bicis disponibles (no alfabéticamente).

Los niveles de prueba que se van a aplicar son los siguientes:

- Pruebas de aceptación. Se definirán siguiendo una estrategia basada en la historia de usuario y sus cinco pruebas (éxito, datos ausentes, datos anómalos, red sin estaciones y error de carga), y se ejecutarán de forma manual sobre la versión debug de la aplicación, que carga los datos desde ficheros JSON locales (`JsonNetworksRepository`), salvo A1.e.2, que se ejecuta sobre la versión release contra el servicio real.
- Pruebas de integración. La estrategia para la definición del orden de las pruebas de integración será jerárquica (ascendente). Se probará:
  - La integración entre el repositorio local (`JsonNetworksRepository`, `Utils.parseNetwork`) y el modelo (`Network`, `Station`, `StationExtra`) con los ficheros del Anexo 1. Técnica de caja negra con JUnit y Robolectric (hace falta un `Context` para leer los recursos raw).
  - La integración entre `StationsPresenter`, `StationUtils` y las clases del modelo, con una vista simulada con Mockito.
  - La integración entre las tres capas: `StationsView`, `StationsPresenter` y `JsonNetworksRepository` con los datos del Anexo 1. Prueba funcional completa definida con la técnica de casos de uso y automatizada con Espresso y Hilt (partiendo de `StationsNavigationUITest`).
- Pruebas unitarias. Se aplicarán a la lógica nueva: formato de las bicis, validación de los datos numéricos, decisión del icono de bicis eléctricas, orden de la lista y presenter de la lista de estaciones. Se usará prueba de métodos con caja negra (partición equivalente y AVL), con JUnit y Mockito.

La aplicación sigue el patrón Modelo-Vista-Presentador. La lista de estaciones ya tiene vista, presenter y contrato (`StationsView`, `StationsPresenter`, `IStationsContract`) y métodos de ayuda en `StationUtils`. Los métodos `formatBikes`, `parseCount` y `hasEbikes` que aparecen en este plan todavía no existen: son una propuesta y deberán ajustarse a los nombres de la implementación final.

A continuación, se muestra una especificación detallada de los casos de prueba a aplicar en cada nivel mencionado anteriormente.

PRUEBAS DE ACEPTACIÓN

En base a la historia de usuario se identifican los siguientes escenarios:

A1. HU: Listar las Estaciones de una Red de Bicicletas

a. Red con estaciones y todos sus datos correctos (caso de éxito, Prueba 1)
b. Red con estaciones con datos ausentes (Prueba 2)
c. Red con estaciones con datos anómalos (Prueba 3)
d. Red sin estaciones (Prueba 4)
e. No se pueden cargar las estaciones (Prueba 5)

Procedimiento común a todos los escenarios: (1) el usuario abre la aplicación y espera a que se cargue la lista principal de redes; (2) selecciona la red indicada en la columna Entrada de la Tabla 2; (3) la aplicación abre la lista de estaciones de esa red; (4) se comprueba que lo mostrado coincide con la columna Resultado, de arriba a abajo; (5) en los escenarios d y e, el usuario pulsa atrás y se comprueba que vuelve a la lista de redes y que la aplicación no se ha cerrado. Con la navegación implementada en la rama, el paso (3) se hace a través de la pantalla "Detalles de la red", pulsando el botón "Ver estaciones" (ver Observaciones). Los datos de cada red están en el Anexo 1 y los resultados esperados en la Tabla 2, que se usa tanto para la ejecución manual como para la automatizada.

Criterios comunes de verificación:

- De cada estación se muestra su nombre y sus bicis como "[bicis_disponibles]/[bicis_totales]". Las bicis totales son las bicis disponibles más los huecos libres (free_bikes + empty_slots).
- Un dato ausente o con valor anómalo se muestra como un guion ("-"). Si no se conocen las bicis disponibles o los huecos, no se pueden calcular las totales y también se muestra "-" (por ejemplo, "5/-"), salvo que el servicio informe de la capacidad de la estación (extra.slots), que se usa entonces como total ("-/12").
- Es anómalo un número negativo, un número con decimales o un valor que no sea un número (texto, booleano). Un dato anómalo de una estación no impide mostrar el resto de estaciones ni el resto de datos de esa estación.
- Se muestra el icono de bicis eléctricas solo si la estación informa de que tiene alguna (extra.ebikes mayor que 0). Con 0, sin dato o con un valor anómalo no se muestra.
- Orden: de más a menos bicis disponibles; si dos estaciones tienen las mismas, por nombre (orden alfabético sin distinguir mayúsculas ni tildes); las estaciones sin dato válido de bicis disponibles van al final, ordenadas también por nombre, y las que no tienen nombre van las últimas de su grupo.
- Los mensajes son exactamente "Esta red no tiene estaciones disponibles" y "No se han podido cargar las estaciones. Comprueba tu conexión".

PRUEBAS UNITARIAS

Se aplican técnicas de clases de equivalencia y AVL para la definición de los casos de prueba de cada método. Además, se deberá garantizar la cobertura de sentencias y de condición/decisión. Las entradas se expresan como el valor del campo en el JSON del servicio; "ausente" indica que la clave no aparece.

Pruebas unitarias de StationUtils

Se aplica prueba de métodos, siendo los casos de prueba definidos para cada método los que se exponen a continuación.

- Método `parseCount(value)`: convierte el valor de un campo numérico (free_bikes, empty_slots, extra.slots, extra.ebikes) en un entero, o null si falta o es anómalo.

Tabla 1. Casos de prueba de la validación de datos numéricos

Identificador	Entrada	Valor esperado
ULE.1a	5 (valor válido)	5
ULE.1b	0 (límite inferior, AVL)	0
ULE.1c	-1 (justo por debajo, AVL)	null (anómalo)
ULE.1d	ausente	null
ULE.1e	null (clave con valor null)	null
ULE.1f	"cinco" (texto)	null (anómalo)
ULE.1g	"5" (número entre comillas)	null (anómalo, es un texto)
ULE.1h	7.5 (decimal)	null (anómalo)
ULE.1i	true (booleano)	null (anómalo)

- Método `formatBikes(freeBikes, emptySlots, slots)`: texto de la columna de bicis.

Identificador	Entrada (free_bikes, empty_slots, extra.slots)	Valor esperado
ULE.2a	5, 7, ausente	"5/12"
ULE.2b	0, 20, ausente (sin bicis, AVL)	"0/20"
ULE.2c	12, 0, ausente (estación llena, AVL)	"12/12"
ULE.2d	0, 0, ausente	"0/0"
ULE.2e	ausente, 4, 12	"-/12"
ULE.2f	ausente, 4, ausente	"-/-"
ULE.2g	5, ausente, ausente	"5/-"
ULE.2h	ausente, ausente, ausente	"-/-"
ULE.2i	-3, 10, ausente (disponibles anómalas)	"-/-"
ULE.2j	2, -1, ausente (huecos anómalos)	"2/-"
ULE.2k	5, 7, 20 (slots no coincide)	"5/12" (manda la suma)

- Método `hasEbikes(station)`: decide si se muestra el icono de bicis eléctricas.

Identificador	Entrada (extra de la estación)	Valor esperado
ULE.3a	{"ebikes": 2}	true
ULE.3b	{"ebikes": 1} (AVL)	true
ULE.3c	{"ebikes": 0} (AVL)	false
ULE.3d	{} (sin ebikes)	false
ULE.3e	extra ausente	false
ULE.3f	{"ebikes": -2} (anómalo)	false
ULE.3g	{"ebikes": "muchas"} (anómalo)	false

- Método `sortByFreeBikes(stations)`: ya existe; hay que añadir el desempate por nombre.

Identificador	Entrada (nombre: bicis disponibles)	Valor esperado (orden)
ULE.4a	A: 3, B: 9, C: 5	B, C, A
ULE.4b	Plaza Mayor: 5, Ayuntamiento: 5 (empate)	Ayuntamiento, Plaza Mayor
ULE.4c	zona Sur: 5, Bulevar: 5 (mayúsculas)	Bulevar, zona Sur
ULE.4d	Ayuntamiento: 5, Árboles: 5 (tilde)	Árboles, Ayuntamiento
ULE.4e	Puerto: ausente, Mercado: 0, Playa: ausente	Mercado, Playa, Puerto
ULE.4f	Hospital: -3, Centro: 4	Centro, Hospital
ULE.4g	sin nombre: 5, Iglesia: 5	Iglesia, sin nombre
ULE.4h	lista vacía (AVL)	lista vacía
ULE.4i	null	lista vacía, sin excepción
ULE.4j	una sola estación (AVL)	la misma estación; la lista original no se modifica

- Método `formatText(text)`: ya existe; se usa para el nombre de la estación.

Identificador	Entrada	Valor esperado
ULE.5a	"Plaza Mayor"	"Plaza Mayor"
ULE.5b	ausente	"-"
ULE.5c	"" (vacío)	"-"
ULE.5d	" " (solo espacios)	"-"

Pruebas unitarias del presenter de la lista

Se prueba `StationsPresenter` aislado, con una vista simulada (mock de `IStationsContract.View`) y un repositorio simulado (mock de `INetworksRepository`) cuyo callback se invoca desde el test, verificando con Mockito las llamadas que hace a la vista. Varios de estos casos ya están en `StationsPresenterTest`.

- Método `init(view, network)`

Identificador	Entrada	Valor esperado
ULE.6a	Red Bici Centro; el repositorio devuelve sus 6 estaciones	showNetworkName("Bici Centro"); showStations con las 6 estaciones en el orden Bulevar, Plaza Mayor, Árboles, Ayuntamiento, Estación Norte, Universidad; showNetworkLocation("Torrelavega, ES · 6 estaciones")
ULE.6b	Red Bici Parcial; el repositorio devuelve sus 5 estaciones	showStations en el orden Mercado, Iglesia, (sin nombre), Playa, Puerto
ULE.6c	El repositorio devuelve una lista vacía	showNoStations; no se llama a showStations
ULE.6d	El repositorio devuelve null	showNoStations; no se llama a showStations
ULE.6e	El repositorio llama a onFailure	showLoadError; no se llama a showStations ni a showNoStations; la cabecera de la red sí se muestra

Identificador	Entrada	Valor esperado
ULE.6f	network null	showLoadError; no se pide nada al repositorio
ULE.6g	Red con id vacío (" ")	showLoadError; no se pide nada al repositorio

PRUEBAS DE INTEGRACIÓN

El orden de las pruebas y los casos de prueba a realizar serían los siguientes:

1. `JsonNetworksRepository` con `Utils.parseNetwork` y el modelo.
   Se llama a `requestStations` con los ficheros del Anexo 1 y se comprueba lo que recibe el callback: Bici Centro, 6 estaciones (ILE.1a); Bici Parcial, 5 estaciones, con null en los campos ausentes (ILE.1b); Bici Errónea, 6 estaciones, con null en los campos anómalos, sin pasar por onFailure (ILE.1c); Bici Sur, lista vacía (ILE.1d); Bici Caída, onFailure porque no existe su fichero (ILE.1e). ILE.1c es el que comprueba que un dato anómalo no hace fallar la carga de toda la red.
2. `StationsPresenter` con `StationUtils`, `Station` y `StationExtra`.
   Los casos de prueba son los mismos definidos como ULE.6x en la sección de pruebas unitarias, aquí renombrados como ILE.6x, pero usando objetos `Station` reales obtenidos al deserializar con Gson los JSON del Anexo 1 (en lugar de construirlos a mano) y el `StationUtils` real.
3. `StationsView` con `StationsPresenter` y `JsonNetworksRepository`.
   Los casos de prueba se definen en base a los escenarios identificados en las pruebas de aceptación, aunque en este caso se automatizarían con Espresso (partiendo de `StationsNavigationUITest`) y deben partir de un estado conocido de los datos. En este caso, el estado inicial es el mostrado en el Anexo 1 y en base a ello, los casos de prueba que se definen son los que se muestran en la Tabla 2. Al no modificar datos, los casos pueden ejecutarse en cualquier orden. A1.e.2 no se automatiza: se ejecuta a mano.

Tabla 2. Casos de prueba de integración funcional completa

Identificador	Entrada	Resultado (de arriba a abajo)
A1.a.1	Seleccionar "Bici Centro" (Torrelavega)	Cabecera: "Bici Centro", "Torrelavega, ES · 6 estaciones" Bulevar: "12/12", con icono Plaza Mayor: "8/12", con icono Árboles: "5/8", sin icono Ayuntamiento: "5/15", con icono Estación Norte: "5/12", sin icono Universidad: "0/20", sin icono
A1.a.2	Seleccionar "Bicing" (Barcelona) (datos reales)	Cabecera: "Bicing", "Barcelona, ES · 545 estaciones" 1ª: MOLL ORIENTAL: "41/41", con icono 2ª: C/ SANCHO DE ÁVILA, 104 - C/ BADAJOZ: "40/50", con icono 3ª y 4ª (empate a 36): AV. LITORAL, 40 (D): "36/52" y PL. PAU VILA (D): "36/36", en ese orden Última: VIA FAVÈNCIA, 186: "0/23", sin icono

Identificador	Entrada	Resultado (de arriba a abajo)
A1.b.1	Seleccionar "Bici Parcial" (Laredo)	Cabecera: "Bici Parcial", "Laredo, ES · 5 estaciones" Mercado: "6/12", sin icono (ebikes desconocido) Iglesia: "5/-", con icono "-" (sin nombre): "3/10", sin icono Playa: "-/-", sin icono Puerto: "-/12", sin icono
A1.b.2	Seleccionar "Bici Norte" (Bilbao)	Cabecera: "Bici Norte", "Bilbao, ES · 5 estaciones" Moyua: "7/12" "-" (sin nombre): "3/12" Guggenheim: "2/-" Abando: "-/-" "-" (nombre en blanco): "-/-" Ninguna muestra icono
A1.b.3	Seleccionar "TUeBICI" (Santander) (datos reales, sin ebikes)	Cabecera: "TUeBICI", "Santander, ES · 42 estaciones" 1ª: Nueva Montaña: "15/20" 2ª: Estación de Gamazo (NE): "11/11" Últimas: Valdenoja: "0/11" y Valdenoja 2 (NE): "0/8" Ninguna muestra icono
A1.c.1	Seleccionar "Bici Errónea" (Reinosa)	Cabecera: "Bici Errónea", "Reinosa, ES · 6 estaciones" Centro: "4/12", con icono Parque: "2/-" (huecos -1), sin icono Colegio: "1/10", sin icono (ebikes "muchas") Estación: "-/-" (bicis "cinco"), sin icono Hospital: "-/-" (bicis -3), sin icono Ría: "-/-" (bicis 7.5, ebikes -2), sin icono No aparece el mensaje de error y la aplicación no se cierra
A1.d.1	Seleccionar "Bici Sur" (Cádiz)	Cabecera: "Bici Sur", "Cádiz, ES" Mensaje: "Esta red no tiene estaciones disponibles" No se muestra la lista ni sus títulos Al pulsar atrás se vuelve a la lista de redes
A1.e.1	Seleccionar "Bici Caída" (Castro Urdiales) (no existe su fichero de detalle)	Cabecera: "Bici Caída", "Castro Urdiales, ES" Mensaje: "No se han podido cargar las estaciones. Comprueba tu conexión" No se muestra la lista La aplicación no se cierra; al pulsar atrás se vuelve a la lista de redes
A1.e.2	Versión release, cargar la lista de redes, activar el modo avión y seleccionar "TUeBICI" (manual)	Mensaje: "No se han podido cargar las estaciones. Comprueba tu conexión" La aplicación no se cierra; al pulsar atrás se vuelve a la lista de redes

ANEXO 1. DATOS DE PRUEBA

Los datos con los que se lanza la aplicación en modo debug, que es necesario conocer para la realización de las pruebas de integración y aceptación, son los de la Tabla 3. Bicing y TUeBICI ya existen en res/raw; Bici Norte está en la rama feature/532377-listarEstaciones; Bici Sur está en los datos de prueba de la historia Ver Información Detallada de Red. Las redes nuevas están preparadas en datos-prueba/estaciones/ (un fichero network_<id>.json por red y networks_estaciones_extra.json con las entradas para networks.json). Bici Caída se añade a networks.json pero sin fichero de detalle, a propósito. Como en el plan de la historia anterior, se recomienda cargarlas desde los recursos de las pruebas, sustituyendo el RepositoriesModule con @TestInstallIn, para no alterar la lista que ve el equipo en debug.

Tabla 3. Estado inicial de los datos de prueba (estaciones)

Red (id)	Estación	free_bikes	empty_slots	extra
bicicentro Bici Centro Torrelavega, ES	Plaza Mayor Estación Norte Ayuntamiento Universidad Bulevar Árboles	8 5 5 0 12 5	4 7 10 20 0 3	ebikes 2 ebikes 0 ebikes 1 ebikes 0 ebikes 3 ebikes 0
biciparcial Bici Parcial Laredo, ES	Mercado Puerto Iglesia Playa (sin name)	6 ausente 5 ausente 3	6 4 ausente ausente 7	ausente slots 12 ebikes 1 {} ebikes 0
bicinorte Bici Norte Bilbao, ES (rama)	Moyua (sin name) Abando Guggenheim " "	7 3 ausente 2 ausente	5 9 ausente ausente 4	ausente en todas
bicierronea Bici Errónea Reinosa, ES	Centro Hospital Estación Parque Colegio Ría	4 -3 "cinco" 2 1 7.5	8 10 6 -1 9 3	ebikes 1 ebikes 0 ebikes 0 ebikes 0 ebikes "muchas" ebikes -2
bicisur Bici Sur Cádiz, ES	stations: [] (lista vacía)			
bicicaida Bici Caída Castro Urdiales, ES	Sin fichero network_bicicaida.json			
bicing / tuebici (existentes)	545 / 42 estaciones reales	todas	todas	Bicing: ebikes en todas TUeBICI: slots, sin ebikes

Observaciones sobre el código actual

Revisando la rama feature/532377-listarEstaciones (que ya incluye las ramas de modelo, carga, datos ausentes, vista, presenter y navegación) se detectan los siguientes puntos que afectan a estas pruebas y que conviene resolver al terminar la historia:

- Formato de las bicis: `StationsArrayAdapter` muestra tres columnas (Bicis, Huecos, Eléct.), como el mock-up, en lugar de "disponibles/totales". Falla en todos los casos de A1.a, A1.b y A1.c.
- Icono de bicis eléctricas: el adaptador escribe siempre "-" en la columna Eléct. ("The current Station model does not provide the number of electric bikes"), aunque `Station.getEbikes()` ya existe. No hay icono.
- Orden: `StationUtils.sortByFreeBikes` ordena de más a menos y deja los null al final, pero no desempata por nombre (mantiene el orden del JSON). Falla ULE.4b-4g; en A1.a.1 Ayuntamiento y Estación Norte saldrían en otro orden.
- Datos anómalos: `free_bikes`, `empty_slots` y `ebikes` son Integer. Un negativo se muestra tal cual ("-3"), y un texto o un decimal hace que Gson lance una excepción: `Utils.parseNetwork` la captura y devuelve null, así que en A1.c.1 aparecería el mensaje de error en vez de la lista. Hay que leer esos campos sin tipar

(JsonElement) o con un TypeAdapter que deje null en los valores anómalos. Lo mismo pasa con el servicio real (Retrofit llama a onFailure).

- Mensaje de error: `StationsView.showLoadError` muestra "No se han podido cargar las estaciones\nComprueba tu conexión a internet", que no coincide con el texto de la Prueba 5. El de red sin estaciones sí coincide.
- Navegación: la lista de estaciones se abre desde "Detalles de la red" con el botón "Ver estaciones", no directamente desde la lista principal como dicen las pruebas; y desde la lista de estaciones, atrás vuelve al detalle (hacen falta dos pulsaciones para llegar a la lista de redes).

Decisiones tomadas por defecto en este plan, a confirmar con el responsable de producto: si faltan las bicis disponibles o los huecos, el total se toma de extra.slots cuando el servicio lo da (necesario para el "-/12" de la Prueba 2), y si no, se muestra "-"; un número entre comillas ("5") se trata como anómalo; el icono depende de extra.ebikes mayor que 0 (hay redes, como Bicing, que dan también has_ebikes); el desempate por nombre no distingue mayúsculas ni tildes; y las estaciones sin nombre se muestran con "-" y van las últimas de su grupo.